

Code review — Variante BAC

Runde de review pe variantele tale de bac, cea mai nouă sus.

Runda 5 — 2026-09-29 — sha 080c9ee

Ai reacționat **în aceeași zi**, la câteva ore după ce ai primit runda 4. Din cele șase constatări , ai atins cinci. E cea mai bună rundă de până acum, și o spun cu dovezile mai jos.

Poarta de compilare:  **TRECUTĂ.** Prima dată în cinci runde. Zero erori, zero warning-uri. Și `main()` apelează efectiv ceva: `rezVar9ex4()`.

Fișier	Runda 4	Acum
<code>temaVar9.h</code>	 4 erori	 reparat
<code>temaVar10.h</code>	 2 erori	 reparat
<code>temaVar11.h</code>	 1 eroare	 reparat
<code>Var12.h</code>	—	 nou, curat
<code>temaVar3.h</code>	 5	 5 (neatins)
<code>temaVar4.h</code>	 2	 2 (neatins)
<code>temaVar5.h</code>	 12	 12 (neatins)
<code>algortimi-utili.h</code>	 2	 2 (neatins)
restul		

Ai și șters `buletinAnalize_2010.pdf` din repo. Bine — era singurul fișier de acolo care n-avea ce căuta într-un repo public.

Ce ai închis

Din runda 4	Stare
B1 — nu compila	✅ ÎNCHIS — build curat, <code>main()</code> apelează <code>rezVar9ex4()</code>
B3 — <code>rezVar10</code> / <code>citVar10</code> în fișierul 11	✅ ÎNCHIS — <code>rezVar11</code> + <code>citVar11</code>
B5 — varianta 9, exercițiul 4 abandonat	✅ ÎNCHIS — rulat: <code>998 997</code> , corect
B4 — <code>f</code> global ca momeală	🟡 pe jumătate — vezi B2
B2 — <code>NR.TXT</code>	🟡 pe jumătate — vezi B1
B6 — două răspunsuri de teorie	🔴 NEATINS — vezi B3

Varianta 9 merită un paragraf separat. Ai terminat cerința 4 corect, cap-coadă:

```
rulat: 998 997          (enunțul cere: 998 997)
```

Ai reparat paranteza, ai scris `Var9Afisare` cu parcurgerea de la 999 în jos, ai oprit-o după al doilea găsit, ai pus și ramura cu `0` dacă nu există două. Și — lucrul care îmi place cel mai mult — ai declarat `int v[1000]={0}` în `rezVar9ex4`. În runda 4 ți-am scris că mărimea vectorului nu călătorește cu el și e pe răspunderea ta. Ai ținut minte și ai dimensionat corect, **și** l-ai inițializat cu zero. Asta nu e reparație mecanică, e înțelegere.

Tot acolo: tabelul de verificare pe care l-ai scris în comentariu (`999 nu · 998 da ct=1 · 997 da ct=2`) e exact cum trebuie făcută o verificare pe hârtie. Mai fă-l.

🔴 Critice

B1 — `NR.TXT` : ai reparat prima jumătate a fix-ului, nu și a doua

```
NR.TXT , temaVar2.h:130 , temaVar11.h:127
```

CUM E ACUM. Fix-ul din runda 4 avea două propoziții. Prima: *fișierul trebuie să conțină intrarea, nu ieșirea*. Ai făcut-o — `NR.TXT` are acum `4 6 3 7 8 1 6 2 7 9 10 8`, exact intrarea din enunțul variantei 11. Rulat, varianta 11 dă acum răspunsul corect **dintr-un test care chiar poate să pice**:

rulat: `4 6 6 7 8 8 8 8 8 9 10 10` (enunțul cere: `4 6 6 7 8 8 8 8 8 9 10 10`)

A doua propoziție era: `NR.TXT` rămâne al variantei 2; varianta 11 își face `numere.txt`. Asta n-ai făcut-o. `NR.TXT` e tot al variantei 11, `numere.txt` nu există, iar varianta 2 e în continuare stricată:

citire3 pe NR.TXT de acum → `12 4 6 3 7 8 1 6 2 7 9 10 8`
enunțul variantei 2 cere → `6 7 7 51 3798`

DE CE. Nu e că n-ai înțeles — se vede din prima jumătate că ai înțeles foarte bine. E că **ai citit fix-ul până la primul lucru de făcut și te-ai oprit acolo**. E a treia oară la rând când se întâmplă exact asta, și de data asta nu mai pot pune pe seama neatenției: în runda 4 ți-am scris ambele jumătăți, una după alta, în aceeași propoziție.

Mecanismul din spate rămâne cel din runda 4: **numele fișierului de date face parte din cerință**, iar două cerințe care cer același nume nu pot trăi în același director. Varianta 2 cere `NR.TXT`. Varianta 11 cere `numere.txt`. Nu e o preferință de-a mea, scrie în enunțuri.

FIX. `numere.txt` primește ce e acum în `NR.TXT`. `NR.TXT` primește înapoi `-3 -10 0 7 -5 7 51 -800 6 3798`. `temaVar11.h:127` deschide `numere.txt`. Apoi rulezi **amândouă** variantele, nu doar pe cea la care lucrezi.

B2 — ai scos momeala din fișierul 11, dar apelurile orfane sunt tot acolo

`temaVar11.h:82` ✓, `temaVar3.h:64`, `temaVar4.h:84`, `temaVar5.h:73`

CUM E ACUM. Ai redenumit `f` → `fVar11ex2`, inclusiv apelul recursiv din corp. Perfect — exact ce trebuia, și ai prins **și** apelul dinăuntru, care e fix ce ți-a scăpat în runda 3.

Dar B4 din runda 4 avea și el două jumătăți: „repară întâi B1 din runda 3, **apoi** redenumeste `f` din `temaVar11.h`”. Ai făcut-o pe a doua. `Var3f` cheamă tot `f(x-1)`, `Varf` tot `f(n,x+1)`, `Var5f` tot `f(a+1,b-2)` — a treia rundă la rând.

DE CE. Aici e o veste bună ascunsă: scoțând `f` din fișierul 11, ai **reduș** numărul de funcții numite `f` din proiect. Cu cât rămân mai puține, cu atât apelurile orfane au mai puține locuri de care să se agațe

tăcut — deci cu atât mai probabil să primești o eroare de compilare cinstită în loc de un rezultat greșit. Mai sunt două: `temaVar1.h:109` și `temaVar2.h:97`.

Dar ele se agață doar dacă fișierele ajung în aceeași compilare — iar `main.cpp` include 3 fișiere din 13. Vezi M1: câtă vreme nu le incluzi, nici erorile cinstite nu apar.

FIX. În `temaVar3.h:64` → `Var3f(x-1)`. În `temaVar4.h:84` și `:86` → `Varf(...)`. În `temaVar5.h:73` → `Var5f(a+1,b-2)`. Apoi rulează-le: `Var3f(26)` trebuie să dea `xyyy`, nu `808y`.

B3 — cele două răspunsuri de teorie sunt neschimbate

`temaVar11.h:14`, `temaVar10.h:30`

CUM E ACUM. Neatinse, cuvânt cu cuvânt: `raaspuns A` și `nr=2*10+2=42`.

DE CE. Bănuiesc ce s-a întâmplat: ai trecut prin review reparând **codul**, iar răspunsurile scrise în comentarii n-au intrat în categoria „de reparat”. E o împărțire pe care o faci fără să-ți dai seama — cod = treabă serioasă, comentarii = notițe.

La bac e fix invers ca pondere. Exercițiile 1–4 sunt grile și pseudocod, **16 puncte din 30** pe subiectul I, și se rezolvă pe hârtie. Codul e un singur exercițiu. Un răspuns greșit lăsat în comentariu e un punct pierdut la fel de real ca un program care nu compilează — cu diferența că pe ăsta nu ți-l semnalează nimeni la examen.

FIX. `temaVar11.h:14` → `c`. Recitește de ce, în runda 4: `==` întoarce adevărul comparației, nu rezultatul scăderii. · `temaVar10.h:30` → `nr = 4 + 2*10 = 24`, și adaugă rândul de rezultat.

B4 — varianta 12: tabelul se oprește la jumătate și are o greșeală în el

`Var12.h:35–43`

CUM E ACUM. Se citesc, în ordine: 12, 7, 354, 9, 630, 0. Tabelul tău are cinci rânduri și se oprește la `3`. Ultimul rând spune `y=1`. Rulat:

x citit	dupa bucla interna	y	
12	1	1	
7	7	17	
354	3	173	<- tu ai scris y=1
9	9	1739	
630	6	17396	

scrie y -> 17396

Două lucruri: rândul lui 354 e greșit ($y = 17 \cdot 10 + 3 = 173$, nu 1), și tabelul se oprește înainte de 9 , 630 și 0 . Răspunsul e 17396 .

DE CE. Greșeala de la rândul 3 e o scăpare de copiere — aveai $y=1$ pe primul rând și l-ai rescris. Dar **de-aia** se oprește tabelul: în clipa în care y a redevenit 1 , următorii pași nu mai duceau nicăieri și ai lăsat-o baltă. O greșeală într-un tabel de urmărire nu rămâne locală — **otrăvește tot ce urmează**, pentru că fiecare rând se calculează din cel dinainte.

De-aia bucla interioară e cheia exercițiului: **cât timp $x > 9$ execută $x = [x/10]$** reduce numărul la **prima lui cifră**. Apoi $y = y \cdot 10 + x$ lipește cifra asta la coada lui y . Deci algoritmul construiește un număr din primele cifre ale numerelor citite: 1, 7, 3, 9, 6 → 17396. Dacă scrii pe marginea tabelului la ce folosește fiecare buclă, rândurile se verifică singure.

FIX. Reia tabelul de la rândul lui 354 și du-l până la 0 . Răspunsul de scris: 17396 .

Restanțe — runda 3, a treia oară

Fișierele n-au fost deschise nici de data asta. `main.cpp` nu le include, deci compilatorul nu ți le-a arătat.

#	Unde	Ce
B1/r3	temaVar3.h:64 , temaVar4.h:84 , temaVar5.h:73	apeluri recursive rămase pe f (vezi B2)
B2/r3	temaVar3.h:63 , temaVar5.h:73	'x' și '*' cu apostrof tipografic (U+2019)
B3/r3	temaVar4.h:124	enunțul cere diagonala secundară, codul face principala
B4/r3	temaVar5.h:166–170	construireVar5() / afisareVar5() chemate fără argumente; intj la :158
B5/r3	temaVar9.h:57	echivalența logică: ai d , corect e b

🟡 Importante

M1 — main.cpp:2–4 : include 3 fișiere din 13. Ai adăugat temaVar9.h — de-aia ai putut rula varianta 9, și de-aia a mers. Dar asta e exact motivul pentru care poarta de compilare trece acum cu 21 de erori rămase în proiect: **compilatorul nu vede ce nu incluzi**. Un build verde pe 3 fișiere din 13 nu e un build verde.

Adaugă-le pe toate. O să vezi dintr-o dată toate restanțele rundei 3 — asta e ideea.

M2 — temaVar2.h:130 : citire3 tot nu sortează. Cerința spune „în ordine crescătoare”. Semnalat în runda 4, neatins.

M3 — restanțele 🟡 din rundele 3 și 4: 7**** cu patru stele deși tabelul tău are cinci rânduri (temaVar5.h:84 , rulat: 7*****) · Var2f tot goală (temaVar2.h:197) · algortimi-utili.h tot nu compilează.

Cleanup

C1 — `Var12.h` iese din convenția ta de nume. Celelalte douăsprezece se cheamă `temaVarNN.h`.
Ăsta e `Var12.h` — exact greșeala pe care ai reparat-o săptămâna trecută redenumind `Var9.h` în `temaVar9.h`. Redenumește-l acum, cât are 51 de linii.

C2 — `temaVar1.h:9` : răspunsul e tot `B`. A cincea rundă. Același exercițiu e rezolvat corect în `temaVar9.h:16`.

C3 — `bin/`, `obj/`, `VARIANTE-BAC.layout` și `.depend` tot comise, tot fără `.gitignore`. A cincea rundă. `bin/Debug/VARIANTE-BAC.exe` și `obj/Debug/main.o` s-au schimbat și în commit-ul ăsta — sunt fișiere pe care le regenerează compilatorul, n-au ce căuta în istoric. Trei rânduri într-un `.gitignore` și subiectul se încheie definitiv.

C4 — commit-urile sunt tot semnate `KERNEL05-PC\Administrator`.

Before / After — B1

	Fișier	Conține	Pentru
Acum	<code>NR.TXT</code>	<code>12 / 4 6 3 7 8 1</code> <code>6 2 7 9 10 8</code>	varianta 11  intrarea corectă
	<code>numere.txt</code>	<i>nu există</i>	—
			varianta 2  rămasă fără datele ei
Corect	<code>NR.TXT</code>	<code>-3 -10 0 7 -5 7</code> <code>51 -800 6 3798</code>	varianta 2
	<code>numere.txt</code>	<code>12 / 4 6 3 7 8 1</code> <code>6 2 7 9 10 8</code>	varianta 11

```
// temaVar2.h:124 – ramane cum e
ifstream f("NR.TXT");

// temaVar11.h:127 – ACUM
ifstream f("NR.TXT");      // ia datele variantei 2

// temaVar11.h:127 – DUPA
ifstream f("numere.txt");   // fiecare cerinta cu fisierul ei
```

Ce ai făcut bine

Verificate rulând codul tău, nu citindu-l.

Ce	Rezultat
Poarta de compilare — build complet, <code>-Wall</code>	zero erori, zero warning-uri ✓
<code>rezVar9ex4()</code> — cele mai mari două numere de 3 cifre lipsă	rulat: <code>998 997</code> ✓
<code>Var11ex3</code> pe intrarea reală , acum că <code>NR.TXT</code> e corect	rulat: <code>4 6 6 7 8 8 8 8 8 9 10 10</code> ✓
<code>int v[1000]={0}</code> — vector dimensionat și inițializat corect	✓ ai aplicat observația din runda 4
<code>fVar11ex2</code> — redenumire cu tot cu apelul recursiv	✓ exact ce-ți scăpa în runda 3
<code>Var12.h</code> exercițiul 1 — numere pare consecutive	<code>d</code> , corect ✓
<code>buletinAnalyze_2010.pdf</code> șters din repo	✓

Trei din șase 🛑 **închise complet, două pe jumătate, unul neatins.** Și, mai important: constatările pe care le-ai închis le-ai închis **bine** — varianta 9 nu e peticită, e rezolvată, cu vectorul dimensionat corect și cu tabel de verificare scris de tine.

Diagnosticul rundei: când te apuci de ceva, îl faci ca lumea. Problema nu mai e calitatea — e **unde te oprești**. De trei runde, fix-urile cu două jumătăți se închid pe prima.

Q&A — trei întrebări

1. Build-ul e verde, dar în proiect au rămas 21 de erori de compilare, în patru fișiere. Cum e posibil?
Ce trebuie să faci ca să le vezi?
 2. Ai pus în `NR.TXT` intrarea corectă a variantei 11 și varianta 11 merge. De ce, cu aceeași mișcare, varianta 2 a rămas stricată? Câte fișiere de date ar trebui să aibă proiectul tău pentru cele două cerințe?
 3. În tabelul de la varianta 12 ai scris `y=1` pe rândul lui `354`, unde trebuia `173`. De ce crezi că tabelul s-a oprit exact acolo, în loc să continue greșit până la capăt?
-

Ce urmează

1. `main.cpp` include toate cele 13 fișiere. Prima mișcare, înaintea oricărei reparații — ca să vezi ce ai de reparat. Nu mai scrie nimic nou până n-o faci.
 2. **Restanțele rundei 3**, în ordinea din tabel. Sunt aceleași de trei runde.
 3. **B1** — `numere.txt` separat, `NR.TXT` înapoi la varianta 2. A doua jumătate.
 4. **B3 și B4** — cele două răspunsuri de teorie, tabelul variantei 12 până la `17396`.
 5. `.gitignore` cu `bin/`, `obj/`, `*.layout`, `*.depend`. Trei minute, se închide definitiv.
-

Runda 4 — 2026-09-29 — sha `61e5cf0`

Variantele 10 și 11 scrise de la zero, plus exercițiul 4 din varianta 9. Lucrate pe branch `feat`, pornit dinaintea rundei 3 — deci fără nicio restanță închisă. **Poarta de compilare: PICATĂ**, 3 erori, toate în fișierele livrate atunci.

-  **B1** — nu compila: `citVar10` nedecarat, `rezVar10` redefinit, `Calcul final:` în afara comentariului •  **ÎNCHIS în runda 5** (build curat, `main()` apelează `rezVar9ex4()`)

- **B2** — `NR.TXT` conținea **ieșirea** în loc de intrare (test care nu putea să pice), Și suprascria datele variantei 2 • **rulat: 12 4 6 6 7 ... în loc de 6 7 7 51 3798** • **pe jumătate în runda 5:** intrarea e corectă, varianta 2 tot fără datele ei (runda 5, B1)
- **B3** — `rezVar10 / citVar10` rămase în `temaVar11.h` (fișierul 10 copiat, redenumit pe jumătate) • **✓ ÎNCHIS în runda 5**
- **B4** — `int f(int n)` global în `temaVar11.h` = momeala care ar lega tăcut apelurile orfane din `temaVar3/4/5` • **pe jumătate în runda 5:** `f` redenumit `fVar11ex2` ✓, apelurile orfane neatinsse (runda 5, B2)
- **B5** — varianta 9 ex. 4 începută și abandonată; `if(x>=100)&&x<=999){` • **✓ ÎNCHIS în runda 5 — rulat: 998 997**, cu `v[1000]={0}` dimensionat corect
- **B6** — două răspunsuri de teorie greșite: `temaVar11.h:14` (`a` , corect `c` — `==` întoarce adevărul comparației) și `temaVar10.h:30` (`42` , corect `24`) • **NEATINS** (runda 5, B3)
- **M1** `main.cpp` include 2 din 11 • **M2** `citire3` nu sortează (rulat: `7 7 51 6 3798` vs `6 7 7 51 3798`) • **M3** restante • **runda 3 — toate NEATINSE**
- **C1** `temaVar1.h:9` răspuns `B` • **C2** `bin/ + obj/` • **C3** identitate git • **C4** `.layout` comis — **NEATINSE**

✓ **Corecte prin rulare:** `Var11ex3` (maximul prefixului — soluția eficientă cerută de enunț) și `Var10ex5` (pătrate perfecte pare, dintr-un contor din 2 în 2). Plus `f(23159)=1` , `f(3,1)=9` , `11 brcdbr` , `abacde` , `9/2*2-5=3` , pseudocodul cu 15.

Runda 3 — 2026-09-23 — sha `d1f02aa`

Prima ta reacție la un review pe repo-ul ăsta, în câteva ore. Ai umblat în șase din opt fișiere și ai scris varianta 9 de la zero. Fișiere stricate: de la 4 la 2. Cele cinci coliziuni de nume — toate închise.

`main()` apela efectiv ceva.

- **B1** — funcții redenumite (`f` → `Var3f / Varf / Var5f`), apelurile recursive din corp lăsate pe `f` • se legau tăcut de `f` -urile altor fișiere • **rulat: Var3f(26) → 808y** în loc de `xyyy` • **NEATINS** (runda 4, B1 restante — și repetat la `rezVar10` , runda 4 B3)
- **B2** — `temaVar3.h:63` bara `|` scoasă, `'x'` cu apostrof tipografic lăsat; `temaVar5.h:73` neatinsă • **NEATINS**
- **B3** — `temaVar4.h:124` enunțul reparat, codul lăsat pe diagonala principală • **NEATINS**
- **B4** — `temaVar5.h` semnături reparate, apelurile din `rezolvareVar5` rămase fără argumente; `intj=1` la :158 • **NEATINS**
- **B5** — `Var9.h:57` echivalență logică: răspuns `d` , corect `b` • **rulat pe toate perechile din [-5,20]: b diferă în 0 cazuri din 676, d în 336** • **NEATINS**

- 🟡 **M1** 7**** cu patru stele (rulat: 7*****) · **M2** main.cpp include 2 din 9 · **M3** Var2f goală · **M4** Var9BAC.TXT gol · **M5** algortimi-utili.h nu compilează — M1, M3, M5 **NEATINSE**; **M2** **agravat** (2 din 11); **M4** ✅ **ÎNCHIS**
- 🟢 **C1** temaVar1.h:9 răspuns B , iar același exercițiu rezolvat corect în Var9.h:16 · **NEATINS**, a patra rundă · **C4** bin/ + obj/ comise · **NEATINS**

✅ **Varianta 9 — 5 din 6 corecte, verificate prin rulare:** Var9sub(3,5) → 15 10 5 (prima funcție rulată de tine, din main) · Var9f(1213111,1) → 3 · strcpy/strcat → abcd123efg · pseudocod n=103456→963 · pseudocod n=1239→1 · citire struct → a. **B6 din runda 2 închis:** rulat pe bacalaureat → baAcaAlaAuUreEaAt .

Runda 2 — 2026-09-23 — sha 7796154

1137 de linii noi, variantele 5–8. Patru din zece fișiere nu compilau; main() era gol.

- ✅ **B2** — ghilimele tipografice + bara | · **închis în** temaVar4.h ; în temaVar3.h doar pe jumătate și în temaVar5.h deloc (runda 3, B2)
- ✅ **B3** — cif definit de două ori în temaVar3.h · **închis** (redenumit cifEx4 ; a rămas enunțul dublat — runda 3, C5)
- ✅ **B4** — 5 coliziuni de nume între fișiere · **TOATE ÎNCHISE**, zero erori de redefinire (dar redenumirea a deschis B1 din runda 3)
- 🟡 **B5** — temaVar4.h rezolva cerința variantei 3 · **INVERSAT**: enunțul e corect acum, codul nu (runda 3, B3)
- ✅ **B6** — cuvântNou fără '\0' · **ÎNCHIS**, rulat curat
- 🟡 **B7** — temaVar5.h , variabile locale folosite ca globale · semnăturile reparate, apelurile nu (runda 3, B4)
- 🟡 **B8** — f(3,17) = 7***** · mecanismul prins, numărătoarea tot greșită (runda 3, M1)
- 🟡 **M1** — main() gol · apelează ceva acum, dar include 2 fișiere din 9 (runda 3, M2)
- 🟡 **M2–M7, C1–C11** — n neinițializat în rez3 · citire3 afișează de două ori · Var2f fără return · algortimi-utili.h · NR.TXT partajat de 3 cerințe · litera de la temaVar1.h:9 · .gitignore · identitatea git — **toate neatinsse**

⚖️ **Fișierul** buletinAnalyze_2010.pdf **din repo:** decizie luată, subiect închis.

Verificate prin rulare și corecte în runda 2: matricea pe coloane (V3), 3798 din NR.TXT , Mare Frig Rosu cu spații multiple, sumele parțiale 15/11/8/1 , 9+36+9=54 , prefixele 10305 1030 103 10 1 , cel mai mic divizor prim, pseudocodul cu k (22), f(12345) = 42 .

Runda 1 — 2026-09-15 — sha 19f5193

Branch `master` , 4 fișiere, 500 de linii. `temaVar3.h` și `temaVar4.h` nu compilau.

-  **B1** — ghilimele tipografice + bara `|` din foaia de examen, în `temaVar3.h:63,67` și `temaVar4.h:82,87` · **NEATINS** (runda 2, B2)
-  **B2** — `rezolvare5` sărea consoanele și nu dubla vocala · **ÎNCHIS** (logica e corectă acum; a rămas terminatorul de șir — runda 2, B6)
-  **B3** — `varianta-1.h:10` , condiția gândită corect dar bifată litera `b` în loc de `d` · **NEATINS** (runda 2, C2)
-  **B4** — șapte cerințe fără rezolvare · **2 din 7 închise** (numerele divizibile cu n din `bac.txt`  rulat, și numerele cu cel mult 2 cifre din `NR.TXT` ); una rezolvată greșit (matricea cu diagonala secundară — runda 2, B5); patru rămân (runda 2, C1)
-  **M1** — `temaVar4.h` fără `#include <iostream>` · **NEATINS**
-  **M2** — `int A[100][100]` pentru `n,m ≤ 10` · **NEATINS** (runda 2, C5)
-  **M3** — `strlen()` recalculat în condiția buclei · **NEATINS**, acum în 3 locuri (runda 2, C6)
-  **M4** — `main.cpp` nu apelează nimic · **NEATINS**, acum `main()` e complet gol (runda 2, M1)
-  **M5** — `varianta-1.h` fără gardă de includere · **NEATINS** (fișierul se numește acum `temaVar1.h` — runda 2, C3)
-  **C1** — `minim` cu o ramură în plus · **NEATINS** (runda 2, C7)
-  **C2** — `bac1.txt` în loc de `bac.txt` · **NEATINS** (runda 2, C8)
-  **C3** — citește `m` înaintea lui `n` · **NEATINS** (runda 2, C8)
-  **C4** — `bin/` și `obj/` comise, fără `.gitignore` · **NEATINS** (runda 2, C9 — și cauza lui B1)

Verificate prin rulare și corecte în runda 1: `f(121,1) → 12345` , `f(12345) → 5310135` , `f(26)`

→ `xyyy` , matricea `min(i,j)` , mijlocul segmentului, `cif(a,b)` , și itemii cu variante de răspuns de la variantele 2, 3 și 4.

